Sorting Performance Analyzer (SPA)

Unit-3 Assignment | Data Structures (ETCCDS202) | School of Engineering & Technology

1. Correctness Verification

All three algorithms were tested on the input list [5, 2, 9, 1, 5, 6]. The expected output is [1, 2, 5, 5, 6, 9]. Each algorithm produced the correct result.

Algorithm	Output	Status
Insertion Sort	[1, 2, 5, 5, 6, 9]	**PASS**
Merge Sort	[1, 2, 5, 5, 6, 9]	**PASS**
Quick Sort	[1, 2, 5, 5, 6, 9]	**PASS**

2. Execution Time Results (milliseconds)

The table below shows execution times for all three algorithms across nine dataset configurations (3 sizes × 3 input types). The same timing method (time.perf_counter) was used for all algorithms. Each array was copied before sorting to ensure a fair comparison. Random seed 42 was used for reproducibility.

n	Input Type	Insertion Sort	Merge Sort	Quick Sort
1000	random	13.166 ms	1.457 ms	0.807 ms
1000	sorted	0.064 ms	0.991 ms	37.533 ms
1000	reverse	50.401 ms	1.093 ms	30.471 ms
5000	random	439.543 ms	18.525 ms	4.907 ms
5000	sorted	0.435 ms	6.721 ms	1102.572 ms
5000	reverse	837.161 ms	8.046 ms	689.470 ms
10000	random	1747.039 ms	17.516 ms	10.189 ms
10000	sorted	1.229 ms	14.452 ms	3978.427 ms
10000	reverse	3328.357 ms	13.219 ms	2888.771 ms

** Highlighted cells indicate Quick Sort worst-case behaviour on sorted/reverse inputs.*

3. Analysis and Theory Comparison

Random Input

On random data, Merge Sort and Quick Sort both perform significantly better than Insertion Sort, which is expected given their O(n log n) average case versus O(n^2) for Insertion Sort. At n=10000, Insertion Sort took approximately 1747 ms while Merge Sort and Quick Sort finished in 17 ms and 10 ms respectively — roughly a 100x difference. Quick Sort outperformed Merge Sort on random data due to lower constant factors and better cache locality.

Sorted Input

Sorted input reveals the best case for Insertion Sort (near O(n) behaviour — only 1.2 ms at n=10000) because the inner while loop exits immediately on each pass. Quick Sort, however, degrades severely: with last-element pivot selection on an already sorted array, every partition produces one empty sub-array and one of size n-1, giving $O(n^2)$ behaviour. This was observed clearly — Quick Sort took 3978 ms at n=10000 on sorted input versus 14 ms for Merge Sort.

Reverse-Sorted Input

Reverse-sorted input is the worst case for Insertion Sort — every element must travel the full length of the sorted prefix, producing $O(n^2)$ comparisons and shifts (3328 ms at n=10000). Quick Sort also suffers because the last element is always the minimum, again creating maximally unbalanced partitions (2888 ms). Merge Sort is unaffected by input order and consistently runs in O(n log n) time across all three input types.

$O(n^2)$ vs O(n log n) — Observed vs Theoretical

Scaling Insertion Sort from n=1000 to n=10000 (10x increase) on random input increased runtime by approximately 130x (13 ms to 1747 ms), close to the expected 100x for $O(n^2)$. Merge Sort scaled from 1.5 ms to 17.5 ms — about 12x, which matches the expected n log n ratio of roughly 13x for a 10x increase in n. These results closely confirm the theoretical complexities.

4. Stability and Memory Properties

Algorithm	Time (avg)	Time (worst)	Stable?	In-place?	Extra Space
Insertion Sort	$O(n^2)$	$O(n^2)$	Yes	Yes	O(1)
Merge Sort	O(n log n)	O(n log n)	Yes	No	O(n)
Quick Sort	O(n log n)	$O(n^2)$	No	Yes	O(log n)

Insertion Sort is stable because equal elements are never swapped past each other — the loop condition uses strict greater-than. Merge Sort preserves order by preferring the left sub-array on ties during the merge step. Quick Sort with Lomuto partitioning is not stable as elements may be swapped across equal values during partitioning.

5. Conclusion

For general-purpose sorting of large random datasets, Quick Sort is the fastest in practice due to cache efficiency, but its $O(n^2)$ worst case on sorted inputs makes it risky without a better pivot strategy (e.g. median-of-three or random pivot). Merge Sort is the safest choice when guaranteed O(n log n) performance is required and extra memory is available. Insertion Sort, despite being $O(n^2)$, is optimal for small arrays or nearly-sorted data, which is why it is used as the base case in hybrid algorithms like Timsort.

References: Introduction to Algorithms (CLRS), 3rd Ed. — Cormen et al. | Python docs: time.perf_counter()